

21 de octubre de 2025

Memcached

Asignatura:

Administración de sistemas gestores
de bases de datos

Diego Vargas Gómez de Requena



Índice

Explicación del SGBD.....	3
Instalación y configuración para uso remoto.....	6
Uso básico y explicación del funcionamiento.....	8
Funcionamiento de Memcached.....	8
Uso básico.....	9
Desde Telnet (modo texto).....	9
Desde un lenguaje (Python).....	11



Explicación del SGBD

Memcached es un sistema de almacenamiento en caché en memoria (RAM) de:

- **Clave-Valor** → Estructura simple de datos (clave: valor).
- **Distribuido** → Usa varios servidores como una sola caché.
- **Alto rendimiento** → Operaciones en memoria, rápidas y ligeras.

El proposito de este SGBD es acelerar aplicaciones web, reduciendo la carga en bases de datos o servicios externos, guardando temporalmente datos ya consultados o procesados.

Su funcionamiento es el siguiente:

- Los datos se guardan como pares: clave → valor.
- Se almacenan en memoria RAM, lo que permite accesos muy rápidos.
- Cuando la memoria se llena, reemplaza los datos menos usados (algoritmo LRU).

Algunas características claves son:

- Muy rápido y ligero (escrito en C).
- Distribuido → se unen varios servidores Memcached como un solo pool de caché.
- No persistente → si se reinicia, se pierden los datos.
- Es usado por grandes plataformas como: Facebook, Twitter, Wikipedia, etc.

Como es un SGBD que no guarda sus datos permanentemente su uso es diferente al de otros tipos:

- Cachear resultados de consultas SQL
- Guardar sesiones de usuario
- Cachear resultados de APIs o plantillas renderizadas.



Memcached no usa cliente como por ejemplo PostgreSQL, si no que usa bibliotecas para que memcached se pueda comunicar con diferentes lenguajes de programación como por ejemplo:

- **Python** → pylibmc, python-memcached, pymemcache
- **PHP** → Memcached, Memcache
- **Node.js** → memjs, memcached
- **Java** → spymemcached, XMemcached
- **Go** → gomemcache
- **Ruby** → dalli

Los nombrados anteriormente no son clientes CLI de Memcached, si no que son librerías / extensiones que usan diferentes lenguajes para comunicarse.

Para pruebas se suele usar telnet, que es una herramienta que permite conectarse a otros equipos a través de la red usando una conexión de texto simple.

Telnet te permite "hablar" directamente con un servidor por medio de comandos escritos, como si se abriera una sesión manualmente.

Por ejemplo, para memcached sería:

```
telnet <ip> 11211
```

Este comando conecta con un servidor memcached (default port 11211) y cuando se conecta se pueden escribir comandos del protocolo memcached, por ejemplo:

```
set saludo 0 60 5
hola!
get saludo
```

Y con esa entrada el servidor responde:

```
VALUE saludo 0 5
hola!
END
```



Más tarde en el apartado de uso de Memcached explicaré los comandos y que es 0.60.5 en los comandos anteriores.



Instalación y configuración para uso remoto

Para instalar Memcached es muy fácil ya que tiene candidato de instalación en Debian13 con apt:

```
sudo apt install memcached -y
```

Ahora pasaré a configurar el SGBD para su uso remoto. Esto es muy sencillo ya que será necesario modificar en el fichero `/etc/memcached.conf` la línea:

```
-l 127.0.0.1
```

por

```
-l 0.0.0.0
```

Después de modificar esto se reiniciará el servicio:

```
sudo systemctl restart memcached.service
```

Y con esta configuración ya se podrá acceder al servidor memcached remotamente, ya que escuchará cualquier IP y no solo la local (127.0.0.1).

Nota: Se puede comprobar con el comando `ss -tln`.

Para realizar la prueba yo usaré el siguiente comando:

```
telnet memcached 11211
```

Nota: En el anterior comando pongo memcached y no la IP, por que lo tengo configurado en el fichero de direcciones estáticas `/etc/hosts` para que el nombre apunte a la IP.



Aclarar que memcached en aplicaciones pequeñas y medianas se suele usar en el mismo servidor ya que así tendrá 0 latencia y configuraciones más simples. Pero en aplicaciones más grandes o distribuidas si se usa en servidores diferentes a los que se tienen que conectar las aplicaciones, esto lo hace más escalable y hace que no se consuma memoria del servidor de la aplicación.

Y otra aclaración es que memcached no tiene autenticación ni cifrado por lo que nunca debería de estar expuesto a Internet directamente, se debería de usar en una red interna entre los distintos servidores que lo usen.



Uso básico y explicación del funcionamiento

Funcionamiento de Memcached

Como ya expliqué antes, memcached es un sistema de caché en memoria, guarda datos temporalmente en la memoria RAM para evitar recalcular o consultar la base de datos cada vez.

El flujo básico es el siguiente:

- La aplicación pide un dato como por ejemplo el perfil del usuario 123.
- Primero busca en memcached.
- Si el dato está en memcached, lo devuelve rápido porque está en la memoria RAM.
- Si no está, la aplicación lo busca en la base de datos, lo guarda en memcached para la próxima vez, y lo devuelve.

A este proceso se le llama cache-aside pattern (leer de la caché, o llenar si no existe).

Lo que almacena memcached son pares clave → valor como por ejemplo:

```
"user:123" → "{ 'nombre': 'Ana', 'edad': 25 }"  
"pagina:home" → "<HTML del home>"  
"producto:456" → "{ 'precio': 99.99 }"
```

Todo esto lo guarda en la memoria RAM, no en el disco, y cuando se llena expulsa los datos menos usados usando el algoritmo LRU (Least Recently Used).



Uso básico

Ahora voy a explicar como se usa con comandos y con código (Python).

Desde Telnet (modo texto)

Para entrar telnet <IP> 11211:

```
telnet memcached 11211
```

Guardar un valor:

```
set saludo 0 60 5  
hola!
```

Esto guarda el valor "hola!" con clave "saludo"

- 0 → flags (normalmente 0)
- 60 → tiempo de expiración (segundos)
- 5 → longitud del valor

El servidor responde:

```
STORED
```

Nota: Las flags se suelen usar para marcar el tipo de dato por ejemplo: 0 sin definir, 1 json, 2 binario, 3 comprimido. Estos datos son para el programador de la aplicación, ya que pueden significar lo que decida en la aplicación.



Leer un valor:

```
get saludo
```

El servidor responde con:

```
VALUE saludo 0 5  
hola!  
END
```

Borrar un valor:

```
delete saludo
```

El servidor responde con:

```
DELETED
```



Desde un lenguaje (Python)

Lo primero que haré será crear un entorno virtual e instalar la librería para usar memcached en un programa Python:

```
python3 -m venv venv
source venv/bin/activate
pip install pymemcache
pip freeze > requirements.txt
```

El programa creado es el siguiente:

```
from pymemcache.client import base
# Conexión al servidor Memcached
client = base.Client(('192.168.122.14', 11211))
# Guardar un valor
client.set('usuario:1', 'Ana')
# Leerlo
valor = client.get('usuario:1')
print(valor)
# Borrarlo
client.delete('usuario:1')
```

Cuando se ejecuta devuelve lo siguiente:

```
b'Ana'
```

Nota: la b significa que el dato es de tipo byte.

En el programa python no se indica el TTL que es la vida útil del dato introducido, por lo que se mantendrá ahí indefinidamente hasta que se necesite eliminar.

Como memcached tiene una cantidad fija de RAM, cuando esta se llena es cuando se eliminan los datos menos usados recientemente.



Para indicar una flag o un TTL a un dato se haría de la siguiente manera:

```
client.set('clave', 'valor', expire=30, flags=0)
```